

Universitatea de Stat din Moldova

Facultatea de Fizică și Inginerie

Infrastructură de măsurare pentru compararea configurațiilor LLM într-un *pipeline* multi-agent

Penzari Marc

TI231

2026

Context și problemă

- **Revânzarea în e-commerce:** extragere -> rescriere -> imagini -> preț -> publicare;
- **LLM-urile** fac automatizarea robustă: citesc pagina ca un om, produc ieșire structurată;
- **Întrebarea principală:** dintre numeroasele LLM-uri (preț, prompt, implementare diferite), cum se compară pe o sarcină fixă și reproductibilă?
- **Două livrabile:** un flux multi-agent funcțional + o infrastructură de măsurare în jurul lui.

Fundamente tehnice

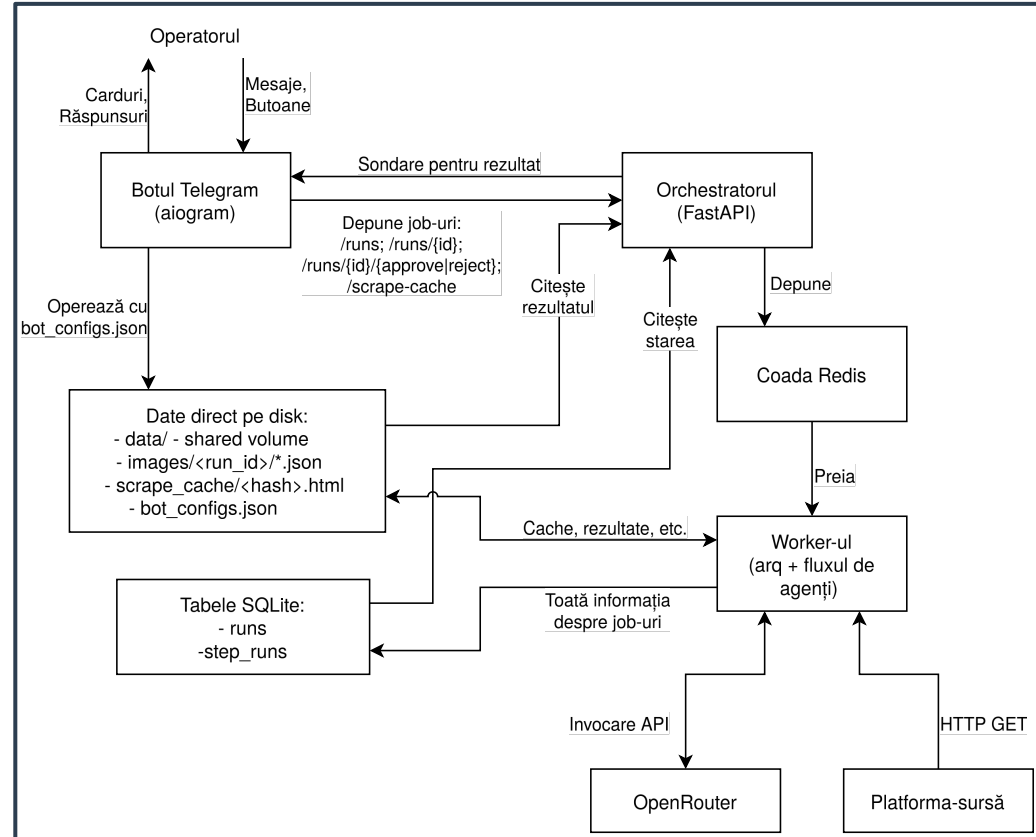
- **Sistem multi-agent:** sarcina spartă în sub-sarcini, fiecare = un agent (un apel LLM cu prompt și contract de I/O);
- **Renunțare la *framework-uri*:** LangChain / LangGraph / AutoGen -> orchestrator propriu de ~200 linii;
- **Extragere structurată:** mod JSON + validare de schemă (Pydantic) cu reîncercare-o-dată la eșec;
- **Web scraping anti-bot:** impersonare TLS (curl_cffi), rezervă Playwright, recuperare manuală folosind om-în-bucă;
- **Interfață asincronă:** bot Telegram (notificări push pentru rulări lungi).

Arhitectura sistemului

- **Patru servicii care cooperează:** bot, orchestrator, worker (Python 3.12) + Redis ca broker;
- **Separare pentru izolarea eșecurilor:** un apel LLM lent în worker nu blochează botul;
- **Orchestrator (FastAPI):** API REST, ciclul de viață al rulării. Worker (**arq**): execută fluxul, înregistrează măsurători;
- Stocare:
 - **runs:** o linie per rulare (mașină de stări: queued -> running -> awaiting_approval -> published/...);
 - **step_runs:** o linie per apel agent (model, latență, token-uri, cost, eroare);
 - Artefacte JSON/imagini pe disc, per rulare.

Pipeline-ul

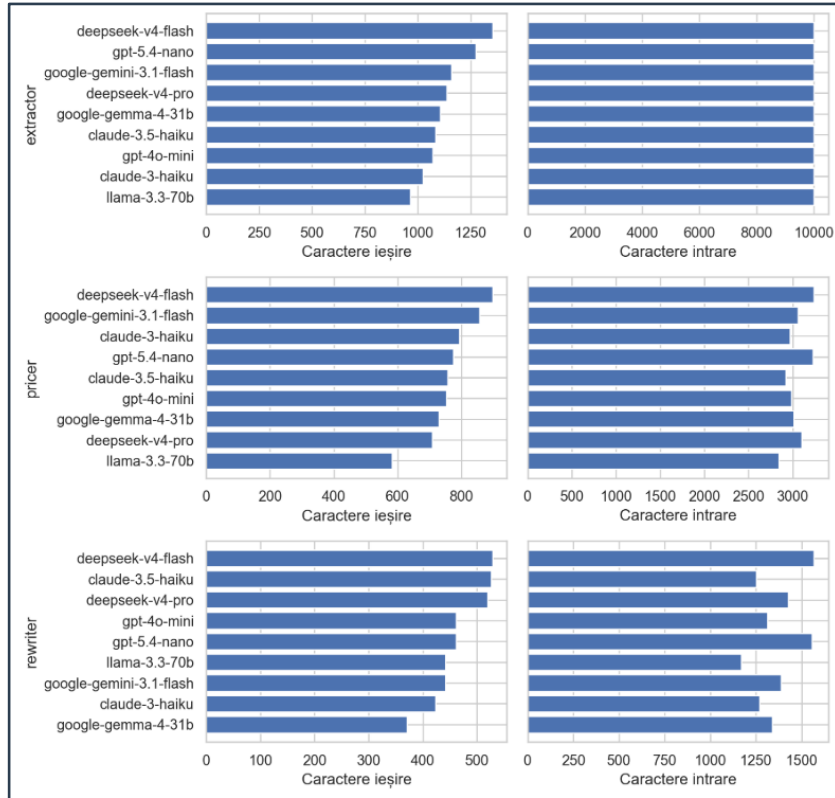
- Șase agenți secvențiali (3 folosesc LLM):
 - *Scraper*;
 - Extractor;
 - Selector imagini;
 - Rescriitor;
 - Evaluator preț;
 - Compozitor calitate;



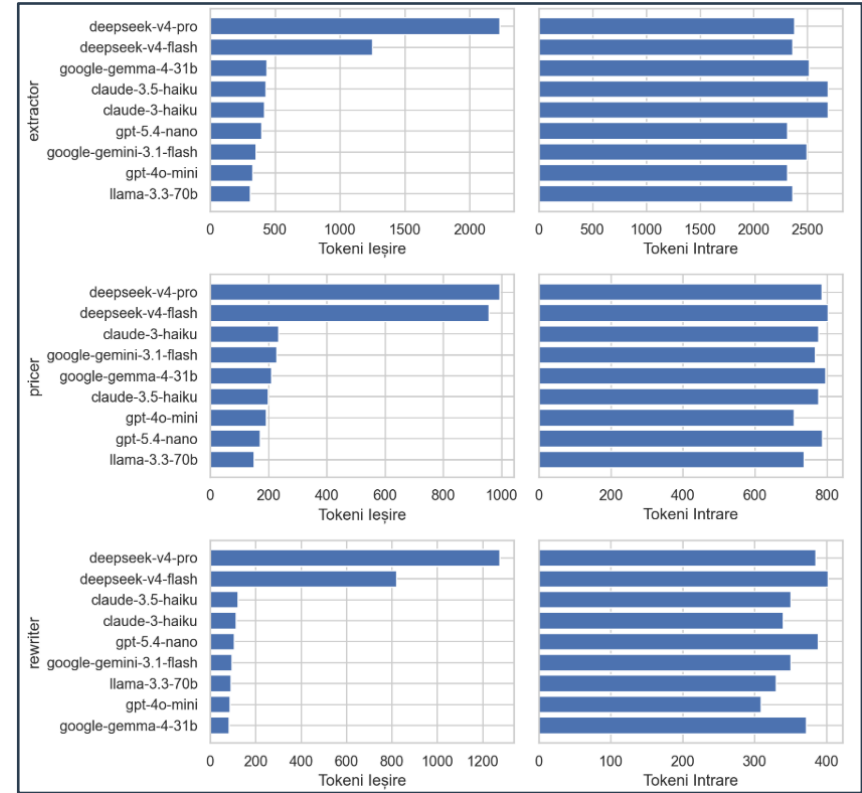
Configurarea experimentului

- **9 configurații:** Claude, DeepSeek, GPT, Gemini, Gemma, Llama;
- Matrice de **189 de rulări** = 9 configurații × 7 anunțuri eBay × 3 repetări.
- **HTML din cache:** fiecare URL descărcat o singură dată;
- **Persistență la nivel de apel:** fiecare apel LLM salvat pe disc.

Rezultate cantitative

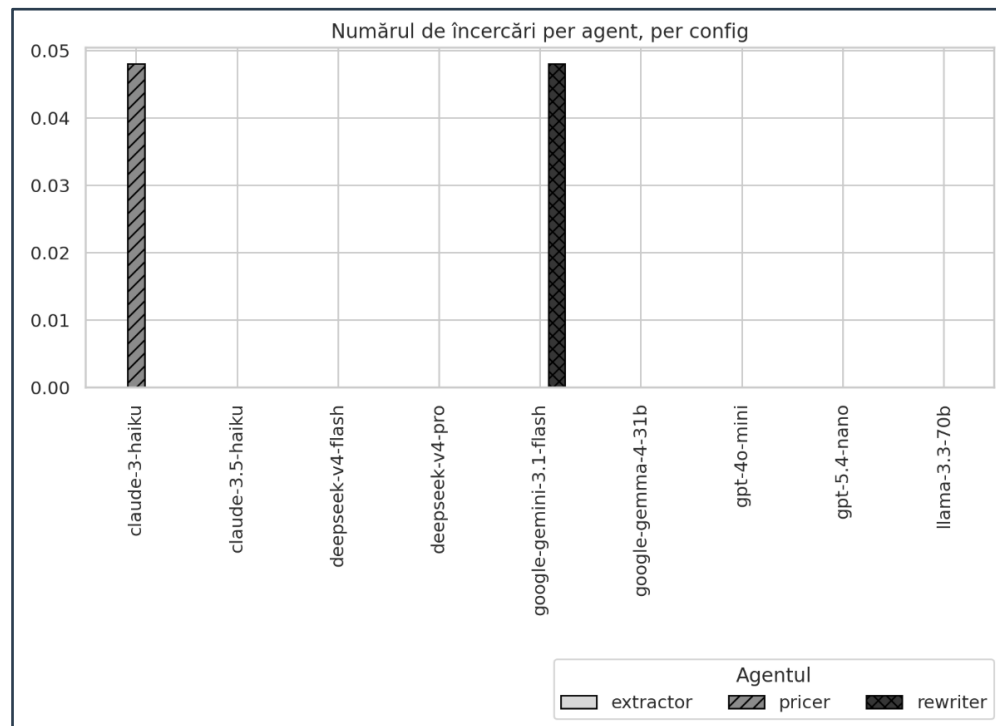


Ieșire/intrare caractere

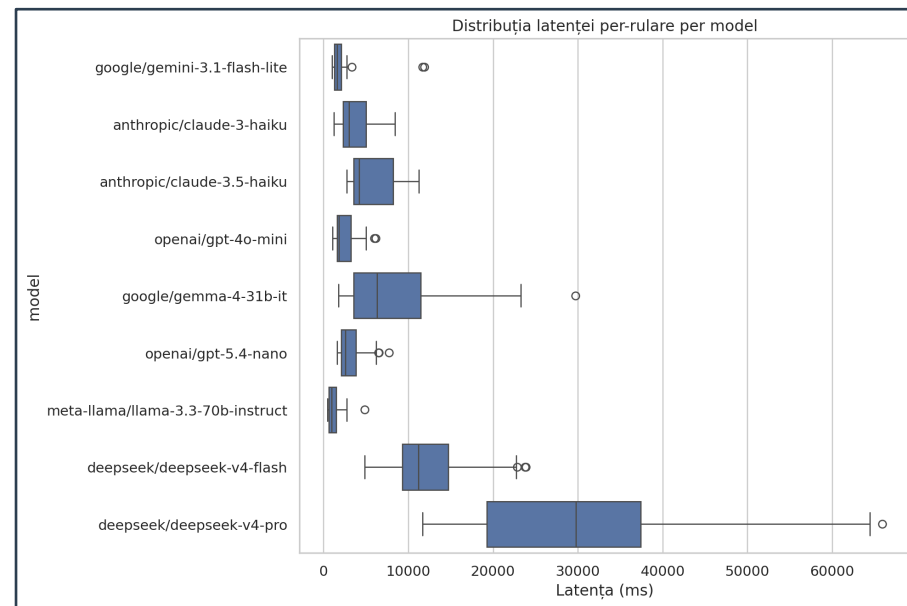


Ieșire/intrare tokeni

Latența și eșecuri



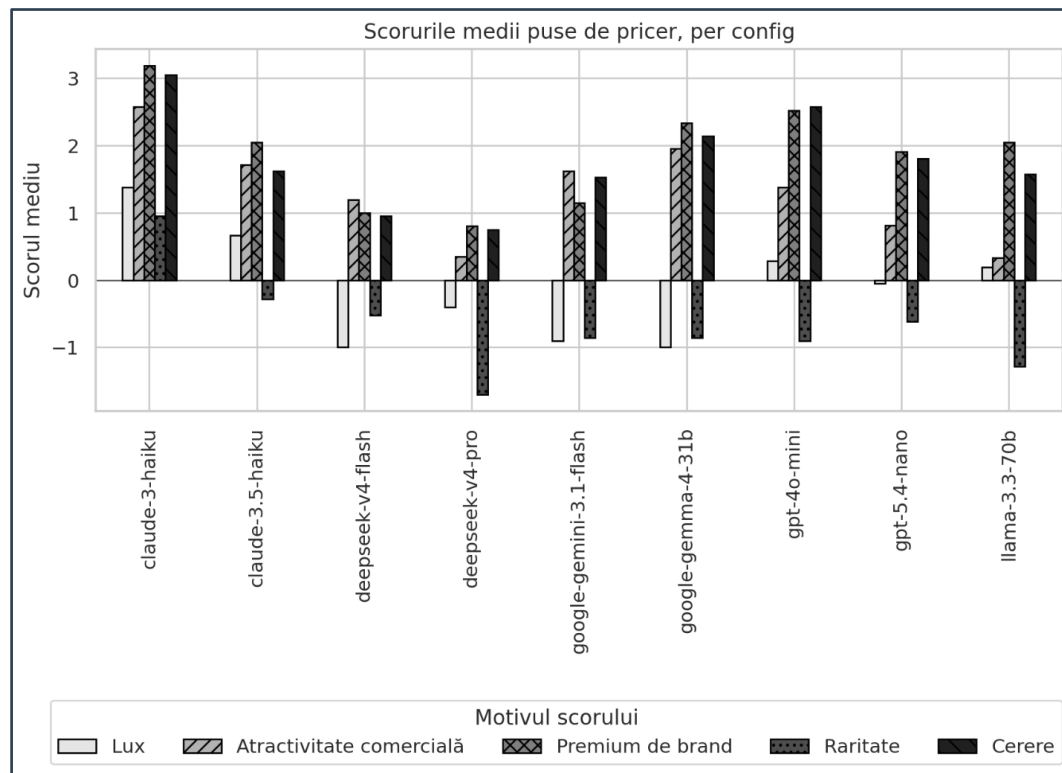
Numărul de încercări



Distribuția latenței

Scoruri prețului

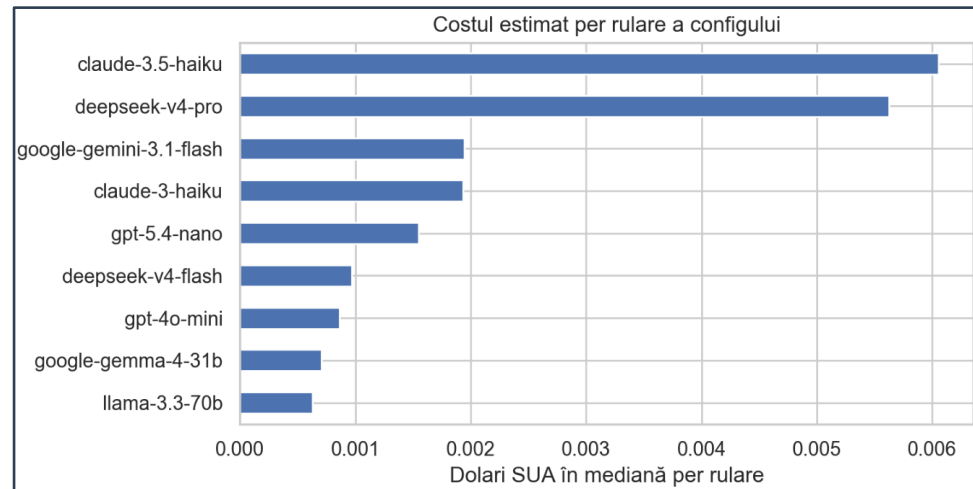
- **Dispoziția de preț:** scoruri pe 5 dimensiuni (-5..+5):
 - Lux;
 - Atractivitatea comercială;
 - Brand;
 - Raritate;
 - Cerere pe piață;



Prețurile de rulare

Configurație	Identificatorul modelului	Prețul-intrare \$/1M	Prețul-ieșire \$/1M
claude-3-haiku	anthropic/claude-3-haiku	\$0,25	\$1,25
claude-3.5-haiku	anthropic/claude-3.5-haiku	\$1,00	\$5,00
deepseek-v4-flash	deepseek/deepseek-v4-flash	\$0,10	\$0,40
deepseek-v4-pro	deepseek/deepseek-v4-pro	\$0,27	\$1,10
google-gemini-3.1-flash	google/gemini-3.1-flash-lite	\$0,10	\$0,40
google-gemma-4-31b	google/gemma-4-31b-it	\$0,10	\$0,30
gpt-4o-mini	openai/gpt-4o-mini	\$0,15	\$0,60
gpt-5.4-nano	openai/gpt-5.4-nano	\$0,05	\$0,40
llama-3.3-70b	meta-llama/llama-3.3-70b-instruct	\$0,13	\$0,39

Prețul per 1M token-uri



Costul per rulare

Concluzii

- **Obiectiv:** nu un model câștigător, ci un *harness* care face LLM-urile măsurabile;
- Două rezultate, de naturi diferite:
 - Pozitiv-nuanțat: rezultatele descriu un LLM sub un anumit amestec de furnizori (rutarea OpenRouter), nu în absolut;
 - Aproape nul: disciplina JSON e prea uniformă (2 reîncercări, 0 eșecuri moale / 189 rulări) pentru a departaja LLM-urile;
- **Limitări:** rutare furnizori, 3 repetări (tendință, nu varianță), 7 anunțuri / o platformă, acuratețea agentului **pricer** imposibil de evaluat;
- **Directii viitoare:** modele locale (axa cloud-vs-local), variația promptului, a doua platformă (Shopify), intrări mai dificile, evaluare reală a prețurilor.